

PLANO DE TESTES DE SOFTWARE

Título: Plano de Testes para GitHub

Componentes do Grupo:

Samuel da Penha Nascimento

Paulo Henrique Alves Vieira

Gabriel Lima Silva Oliveira

1. Introdução e Escolha do Software

1.1 Nome e Versão do Software

O software testado no presente trabalho é o GitHub, plataforma de hospedagem e controle de versão de código, avaliado em sua versão Web (acessível via navegador, sem instalação local).

1.2 Justificativa da Escolha

O GitHub foi selecionado pelos seguintes motivos:

- Possui funcionalidades muito bem definidas e documentadas, facilitando a análise por engenharia reversa;
- É gratuito e de acesso irrestrito, não exigindo licença ou acesso administrativo;
- Apresenta diversas regras de negócio e validações, possibilitando cenários de teste ricos;
- Disponibiliza uma API REST pública e documentada, permitindo a execução de testes funcionais via Postman;
- É amplamente reconhecido pela comunidade acadêmica e pelo mercado, facilitando a contextualização do trabalho;
- Falhas em seus mecanismos de autenticação ou de gerência de código podem comprometer projetos inteiros de software.

1.3 Propósito do Sistema e Público-Alvo

O GitHub é uma plataforma baseada em Git que disponibiliza recursos para controle de versão, colaboração e gerenciamento de repositórios de código-fonte. Entre suas funcionalidades centrais destacam-se: autenticação de usuários, criação e gerenciamento de repositórios públicos e privados, rastreamento de problemas (Issues), revisão de código por Pull Requests e pesquisa global de projetos.

Seus principais usuários são:

- Desenvolvedores individuais e freelancers;
- Empresas de tecnologia e equipes de engenharia de software;
- Organizações de código aberto (open source);
- Estudantes e pesquisadores da área de computação.

2. Análise de Requisitos (Engenharia Reversa)

A seguir são descritas as cinco funcionalidades principais identificadas por observação do comportamento do sistema e de sua API REST pública, sem acesso ao código-fonte interno.

Funcionalidade 1: Login

- Entrada Esperada: e-mail do usuário cadastrado e senha da conta;
- Saída Esperada: usuário autenticado com sucesso; dashboard personalizado carregado;
- Regras de Negócio: os campos de e-mail e senha são obrigatórios; o usuário deve estar previamente cadastrado; as credenciais devem coincidir com os dados registrados.

Funcionalidade 2: Criar Repositório

- Entrada Esperada: nome do repositório e definição de visibilidade (público ou privado);
- Saída Esperada: novo repositório criado e visível no perfil do usuário;
- Regras de Negócio: o nome é obrigatório e não pode ficar em branco; o nome deve ser único dentro do escopo do usuário autenticado; caracteres especiais inválidos não são permitidos no nome.

Funcionalidade 3: Criar Issue

- Entrada Esperada: título da issue;
- Saída Esperada: issue registrada e listada na aba Issues do repositório com número sequencial atribuído automaticamente;
- Regras de Negócio: o título é campo obrigatório; o botão de submissão permanece desabilitado enquanto o título estiver vazio; o usuário deve estar autenticado e ter permissão no repositório.

Funcionalidade 4: Criar Pull Request

- Entrada Esperada: branch de origem (compare) e branch de destino (base);
- Saída Esperada: Pull Request criada com status Open; diferenças entre branches (diff) exibidas para revisão;
- Regras de Negócio: as branches selecionadas devem existir no repositório; deve haver ao menos um commit de diferença entre elas; o sistema deve detectar e alertar sobre conflitos de merge quando presentes.

Funcionalidade 5: Pesquisar Repositórios

- Entrada Esperada: termo de pesquisa (texto livre);
- Saída Esperada: lista de repositórios correspondentes ao termo pesquisado com filtros disponíveis;
- Regras de Negócio: a pesquisa suporta correspondência parcial; filtros por linguagem, estrelas e data de atualização estão disponíveis; resultados são ordenados por relevância por padrão.

3. Estratégia de Testes

A estratégia adotada neste trabalho é baseada em testes funcionais de caixa preta, executados por meio de duas abordagens complementares: requisições diretas à API REST do GitHub via Postman e verificação manual da interface web. Essas duas técnicas são suficientes para cobrir todos os dez casos de teste elaborados na Seção 5.

3.1 Postman — Teste Funcional via API REST

O Postman é a ferramenta principal utilizada em 9 dos 10 casos de teste. Por meio dele, as requisições HTTP são enviadas diretamente aos endpoints da API REST do GitHub, validando o comportamento do sistema sem depender da interface gráfica.

Para autenticar as requisições, é necessário gerar um Personal Access Token (PAT) na conta do GitHub (Settings > Developer settings > Personal access tokens) e incluí-lo no cabeçalho de cada requisição:

- Cabeçalho de autenticação: Authorization: Bearer SEU_TOKEN
- URL base da API: <https://api.github.com>

Os endpoints utilizados nos casos de teste são:

- GET /user — valida autenticação (CT-01 e CT-02);
- POST /user/repos — criação de repositório (CT-03 e CT-04);
- POST /repos/{owner}/{repo}/issues — criação de issue (CT-05 e CT-06);
- POST /repos/{owner}/{repo}/pulls — criação de Pull Request (CT-07 e CT-08);
- GET /search/repositories?q={termo} — pesquisa de repositórios (CT-09 e CT-10).

A validação em cada caso consiste na verificação do código de status HTTP retornado e do conteúdo do JSON de resposta:

- Status 200 OK — requisição bem-sucedida com retorno de dados;
- Status 201 Created — recurso criado com sucesso;
- Status 401 Unauthorized — credenciais inválidas ou token expirado;
- Status 422 Unprocessable Entity — regra de negócio violada (nome duplicado, título vazio, branches idênticas).

3.2 Execução Manual — Teste de Interface (UI)

A execução manual é utilizada exclusivamente no CT-06 (criação de issue sem título), em complemento ao teste via Postman. Enquanto o Postman valida a rejeição pela API no backend (status 422), a execução manual verifica o comportamento correspondente na interface gráfica do GitHub, confirmando que o botão de submissão permanece desabilitado enquanto o campo de título está vazio.

Essa combinação de abordagens é importante porque valida as duas camadas do sistema: a regra de negócio implementada no servidor (backend) e o bloqueio preventivo aplicado na interface (frontend), que são complementares e independentes.

3.3 Resumo — Ferramenta por Caso de Teste

A tabela a seguir sintetiza a ferramenta utilizada em cada caso de teste:

ID	Funcionalidade	Endpoint / Ação	Resultado Validado	Ferramenta
CT-01	Login	GET /user com token válido	Status 200 + JSON do perfil	Postman
CT-02	Login	GET /user com token inválido	Status 401 Unauthorized	Postman
CT-03	Criar Repositório	POST /user/repos com nome único	Status 201 Created	Postman
CT-04	Criar Repositório	POST /user/repos com nome duplicado	Status 422 Unprocessable Entity	Postman
CT-05	Criar Issue	POST /issues com título válido	Status 201 Created	Postman
CT-06	Criar Issue	POST /issues com título vazio + teste na interface	Status 422 (API) + botão desabilitado (UI)	Postman + Manual
CT-07	Pull Request	POST /pulls com branches distintas	Status 201 Created	Postman
CT-08	Pull Request	POST /pulls com branches idênticas	Status 422 Unprocessable Entity	Postman
CT-09	Pesquisa	GET /search/repositories?q=termo existente	Status 200 + total_count > 0	Postman
CT-10	Pesquisa	GET /search/repositories?q=termo inexistente	Status 200 + total_count = 0	Postman

4. Riscos e Prioridades

A tabela a seguir apresenta o mapeamento de riscos com base na matriz clássica Impacto x Probabilidade, determinando a prioridade de esforço de teste para cada funcionalidade crítica do sistema.

Funcionalidade	Risco / Cenário de Falha	Impacto	Probabilidade	Prioridade
Login	Falha na autenticação / acesso negado a usuário legítimo	Alto	Média	Crítico
Criar Repositório	Dados não persistidos / repositório não criado no servidor	Alto	Baixa	Alto
Criar Issue	Texto da issue perdido / issue não registrada no tracker	Médio	Média	Médio
Criar Pull Request	Merge incorreto / conflito não detectado entre branches	Alto	Baixa	Alto
Pesquisar Repositórios	Nenhum resultado retornado / resultados irrelevantes exibidos	Médio	Alta	Médio

Legenda de prioridade:

- Crítico: falha compromete o acesso ao sistema inteiro;
- Alto: falha afeta funcionalidade principal com perda de dados;
- Médio: falha impacta experiência, mas não impede o uso;
- Baixo: falha cosmética ou de baixo impacto operacional.

5. Casos de Teste

Esta seção apresenta o detalhamento dos casos de teste projetados para cobrir os principais fluxos lógicos e cenários de exceção. Foram elaborados 10 casos de teste distribuídos entre as cinco funcionalidades, executados via Postman (API REST) e, no caso do CT-06, também por verificação manual da interface.

ID	Funcionalidade	Cenário	Pré-condição	Passos de Execução	Dados de Entrada	Resultado Esperado	Pós-condição
CT-01	Login	Login com sucesso	Usuário cadastrado e não autenticado	1. Enviar GET /user no Postman com token válido 2. Verificar status HTTP da resposta 3. Conferir retorno dos dados do perfil	Authorization: Bearer TOKEN_VALIDO	Status 200 OK; JSON com dados do perfil retornado	Sessão autenticada confirmada via API
CT-02	Login	Login com credenciais inválidas	Usuário não autenticado	1. Enviar GET /user no Postman com token inválido 2. Verificar status HTTP da resposta 3. Conferir mensagem de erro retornada	Authorization: Bearer TOKEN_INVALIDO	Status 401 Unauthorized; mensagem de erro no JSON	Nenhuma sessão criada
CT-03	Criar Repositório	Criação com nome válido e único	Usuário autenticado com token válido	1. Enviar POST /user/repos no Postman 2. Inserir body JSON com nome único 3. Verificar status e resposta	{ "name": "repo-teste-2026", "private": false }	Status 201 Created; JSON com dados do repositório criado	Repositório disponível no perfil do usuário
CT-04	Criar Repositório	Nome de repositório duplicado	Usuário autenticado; repositório 'repo-teste-2026' já existe	1. Enviar POST /user/repos no Postman 2. Inserir o mesmo nome já existente 3. Verificar status e mensagem de erro	{ "name": "repo-teste-2026", "private": false }	Status 422 Unprocessable Entity; mensagem indicando nome já utilizado	Nenhum repositório criado
CT-05	Criar Issue	Issue criada com título válido	Usuário autenticado; repositório existente com Is-	1. Enviar POST /repos/{owner}/{repo}/issues no Postman 2. Inserir body	{ "title": "Bug na autenticação SSO" }	Status 201 Created; JSON com número sequencial da issue	Issue disponível e atribuível no repositório

ID	Funcionalidade	Cenário	Pré-condição	Passos de Execução	Dados de Entrada	Resultado Esperado	Pós-condição
			sua habilidade	com título válido 3. Verificar status e número da issue			
CT-06	Criar Issue	Tentativa de criar issue sem título	Usuário autenticado; repositório existente com Issues habilitadas	Via API: 1. Enviar POST /repos/{owner}/{repo}/issues com título vazio 2. Verificar status Via interface: 1. Acessar aba Issues 2. Clicar em New issue 3. Deixar título em branco 4. Tentar submeter	API: { "title": "" } Interface: título vazio	API: Status 422 Unprocessable Entity Interface: botão Submit desabilitado	Nenhuma issue criada em ambos os casos
CT-07	Pull Request	PR criada com branches válidas e com diferenças	Usuário autenticado; branches 'main' e 'feature/login' com commits distintos	1. Enviar POST /repos/{owner}/{repo}/pulls no Postman 2. Inserir branches de origem e destino 3. Verificar status e diff retornado	{ "title": "PR de teste", "head": "feature/login", "base": "main" }	Status 201 Created; PR com status Open e diff exibido	PR disponível para revisão e merge
CT-08	Pull Request	PR com branches sem diferenças	Usuário autenticado; branches 'main' e 'copia-main' idênticas	1. Enviar POST /repos/{owner}/{repo}/pulls no Postman 2. Selecionar branches idênticas 3. Verificar resposta da API	{ "title": "PR vazia", "head": "copia-main", "base": "main" }	Status 422 Unprocessable Entity; mensagem indicando ausência de diferenças	Nenhuma PR criada
CT-09	Pesquisa	Pesquisa por repositório existente	Usuário autenticado	1. Enviar GET /search/repositories?q=facebook/react no Postman 2. Verificar status 3. Conferir total_count e itens retornados	q=facebook/react	Status 200 OK; total_count > 0; repositório facebook/react na lista	Resultados exibidos ordenados por relevância

ID	Funcionalidade	Cenário	Pré-condição	Passos de Execução	Dados de Entrada	Resultado Esperado	Pós-condição
CT-10	Pesquisa	Pesquisa por termo inexistente	Usuário autenticado	1. Enviar GET /search/repositories?q=xyzxyz_naoexiste_abc123 no Postman 2. Verificar status 3. Conferir total_count e lista de itens	q=xyzxyz_naoexiste_abc123	Status 200 OK; total_count = 0; lista de itens vazia	Nenhum resultado exibido

6. Cronograma e Alocação

6.1 Estimativa de Tempo de Execução

A execução completa do ciclo de testes foi estimada em 4 dias úteis:

- Dia 1 — Engenharia Reversa e Planejamento: levantamento de funcionalidades, definição de escopo e elaboração dos casos de teste (estimativa: 4 horas);
- Dia 2 — Configuração do Postman e execução dos casos CT-01 a CT-06: geração do token, configuração dos endpoints e registro dos resultados (estimativa: 3 horas);
- Dia 3 — Execução dos casos CT-07 a CT-10 e verificação manual do CT-06: conclusão dos testes e registro dos resultados (estimativa: 2 horas);
- Dia 4 — Consolidação e Relatório Final: registro dos defeitos encontrados, redação da conclusão e preparação da apresentação (estimativa: 3 horas).

6.2 Divisão de Responsabilidades

- **Samuel da Penha Nascimento:** Seções 1 e 2 (Introdução e Engenharia Reversa); execução dos casos de teste CT-01 e CT-02 (Login) e CT-05 e CT-06 (Criar Issue) via Postman e verificação manual; registro de defeitos identificados durante a elaboração do plano.
- **Paulo Henrique Alves Vieira:** Seções 3 e 4 (Estratégia de Testes e Matriz de Riscos); execução dos casos de teste CT-03 e CT-04 (Criar Repositório) e CT-07 e CT-08 (Pull Request) via Postman; formatação e revisão final do documento.
- **Gabriel Lima Silva Oliveira:** Seções 6 e 7 (Cronograma e Conclusão); execução dos casos de teste CT-09 e CT-10 (Pesquisa) via Postman; elaboração da tabela de riscos e dos critérios de aceitação.

7. Conclusão e Critérios de Aceitação

7.1 Critérios de Aceitação — Definição de Software Aprovado

O GitHub será considerado aprovado neste ciclo de testes caso atenda aos seguintes critérios:

- 100% dos casos de teste com prioridade Crítica e Alta executados com resultado de aprovação;
- Ausência total de defeitos que causem interrupção inesperada do fluxo principal de uso;
- Todas as cinco funcionalidades principais retornando os códigos de status HTTP esperados via API;
- Nenhuma vulnerabilidade crítica de autenticação identificada (CT-01 e CT-02 aprovados);
- Taxa de sucesso nos testes funcionais igual ou superior a 90%.